

CS 577 — Deep Learning — Midterm — Solutions

- **Do not turn the page until instructed.**
- Write your name and A# in the upper right corner.
- Read all of the problem statements. Make sure you respond to the underlined items.
- No electronic device of any kind.
- This is a closed-book, closed-notes exam.
- One cheatsheet allowed, front-and-back. May be written or printed.
- There are 4 problems for a total of $12 + 24 + 28 + 16 = 80$ points. Plan accordingly.
- You have 90 minutes total.
- Throughout this exam, the training data set denoted by $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)} \in \mathbb{R}^d$ while the training labels are denoted by $y^{(1)}, \dots, y^{(N)} \in \mathbb{R}/\{\pm 1\}/\{1, \dots, K\}$ for regression/binary/multiclass classification, respectively.
- Models are denoted by $f(\mathbf{x}; \boldsymbol{\theta})$.

1 Perceptron

[6] **point(s)** — **part a:** Write the pseudocode for the binary perceptron algorithm where $T > 1$ is the number of iterations ran before stopping. Use lower case t for the iteration counter starting at $t = 1$. Use $\mathbf{w} \in \mathbb{R}^d$ to denote the classifier and assumed that it is initialized to zero. There is no bias term. Your main loop should have a “IF-THEN” statement.

Input: $(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots$, time T

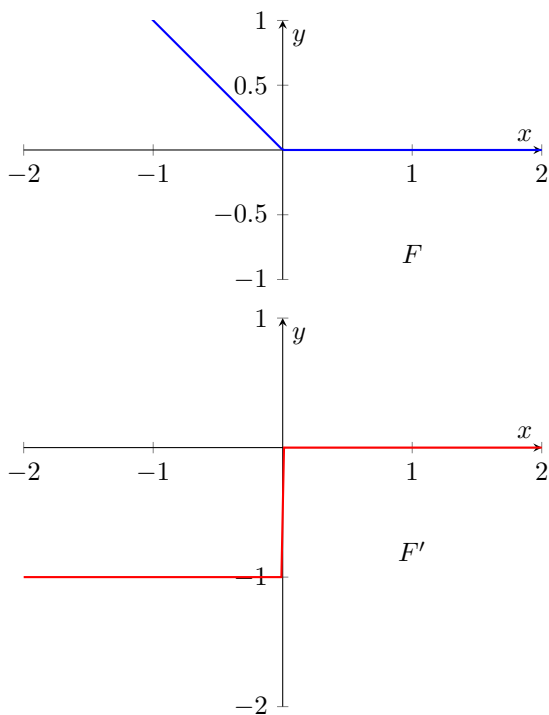
1. Initialize $\mathbf{w} = \mathbf{0}$.
2. For $t = 1, 2, \dots, T$
 - (a) If $y^{(t)} \mathbf{w}^\top \mathbf{x}^{(t)} > 0$, then $\mathbf{w} \leftarrow \mathbf{w}$,
 - (b) Else, then $\mathbf{w} \leftarrow \mathbf{w} + y^{(t)} \mathbf{x}^{(t)}$.

Output: $\mathbf{w}$

[6] point(s) — **part b:** Sketch the plot of the graph of the function $F : \mathbb{R} \rightarrow \mathbb{R}$ defined by $F(u) := \max\{0, -u\}$ and its derivative F' . Note: F' is not differentiable at $u = 0$. Specify the value of $F'(0)$ explicitly so that the stochastic gradient descent (SGD) update

$$\mathbf{w} \leftarrow \mathbf{w} - \nabla_{\mathbf{w}} F(y^{(t)} \mathbf{w}^\top \mathbf{x}^{(t)})$$

is equivalent to that of the (binary) perceptron algorithm.



If part a pseudocode line 2.a has $>$, then $F'(0) = -1$
 otherwise if $\geq$, then $F'(0) = 0$

2 Loss function and conditional probability

[6] **point(s) — part a:** In class, we derived the mean square error (MSE) $J(\boldsymbol{\theta}) := \frac{1}{N} \sum_{i=1}^N (y^{(i)} - f(\mathbf{x}^{(i)}; \boldsymbol{\theta}))^2$ assuming that the labels follows the Gaussian distribution $p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{1}{2} \frac{(y^{(i)} - f(\mathbf{x}^{(i)}; \boldsymbol{\theta}))^2}{\sigma^2}\right)$. Explain step-by-step how $J(\boldsymbol{\theta}) := \frac{1}{N} \sum_{i=1}^N |y^{(i)} - f(\mathbf{x}^{(i)}; \boldsymbol{\theta})|$, the **mean absolute error** (MAE), can be derived from the label distribution (called the Laplace distribution)

$$p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta}) = \frac{1}{2\sigma} \exp\left(-\frac{|y^{(i)} - f(\mathbf{x}^{(i)}; \boldsymbol{\theta})|}{\sigma}\right).$$

Start with the likelihood (the expression involving a “ $\prod_{i=1}^N$ ”). If necessary, drop/add constants (multiplicative or additive) that do not depend on $\boldsymbol{\theta}$. State each drop/add explicitly.

Likelihood

$$L(\boldsymbol{\theta}) = \prod_{i=1}^N p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta}) = \prod_{i=1}^N \frac{1}{2\sigma} \exp\left(-\frac{|y^{(i)} - f(\mathbf{x}^{(i)}; \boldsymbol{\theta})|}{\sigma}\right).$$

Take log

$$\log L(\boldsymbol{\theta}) = \sum_{i=1}^N \log\left(\frac{1}{2\sigma}\right) - \frac{|y^{(i)} - f(\mathbf{x}^{(i)}; \boldsymbol{\theta})|}{\sigma}.$$

Drop the constant terms $\log(2\sigma)$, which do not depend on $\boldsymbol{\theta}$

$$\log L(\boldsymbol{\theta}) = -\frac{1}{\sigma} \sum_{i=1}^N |y^{(i)} - f(\mathbf{x}^{(i)}; \boldsymbol{\theta})|.$$

Drop σ , multiply by $-1/N$:

$$J(\boldsymbol{\theta}) := \frac{1}{N} \sum_{i=1}^N |y^{(i)} - f(\mathbf{x}^{(i)}; \boldsymbol{\theta})|,$$

[6] point(s) — part b: Suppose that the relationship between the sample (input) $\mathbf{x}$ and the label (output) y is governed by

$$p(y | \mathbf{x}) = \frac{1}{2\sigma} \exp\left(-\frac{|y - f(\mathbf{x}; \boldsymbol{\theta})|}{\sigma}\right) \quad (1)$$

Let $\sigma = 1$, $\mathbf{w} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ and $b = 3$. Suppose that $\boldsymbol{\theta} = \begin{bmatrix} \mathbf{w} \\ b \end{bmatrix}$ and $f(\mathbf{x}; \boldsymbol{\theta}) = \mathbf{w}^\top \mathbf{x} + b$. We can simulate y given $\mathbf{x}$ using `np.random.laplace` instead of `np.random.randn` as below:

```
import numpy as np
N = 100000
np.random.seed(42)
w = np.array([0, 1])
b = 3
x = np.vstack([[1, -2] for _ in range(N)])
eps = np.random.laplace(0, 1, N)
y = x@w + b + eps
n1 = np.sum((y >= 1 - 0.1) & (y <= 1 + 0.1))
n2 = np.sum((y >= (1 - np.log(5)) - 0.1) & (y <= (1 - np.log(5)) + 0.1))
print(n1/n2)
```

Based on the information given, make an estimate of the output of the code block using Eqn. (1). Briefly justify your answer.

Answer key: From the code, $\mathbf{x} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$. Plugging in $\mathbf{w}$ and b :

$$f(\mathbf{x}; \boldsymbol{\theta}) = 0 \cdot 1 + 1 \cdot (-2) + 3 = 1.$$

1. `n1` counts how many points are near 1. Thus for $y = 1$:

$$p(1 | \mathbf{x}) = \frac{1}{2} \exp(-|1 - 1|) = \frac{1}{2} \exp(0) = \frac{1}{2}.$$

2. `n2` counts how many points are near $1 - \log(5)$. Thus for $y = 1 - \log(5)$:

$$p(1 - \log(5) | \mathbf{x}) = \frac{1}{2} \exp(-|(1 - \log(5)) - 1|) = \frac{1}{2} \exp(-\log(5)) = \frac{1}{2 \times 5}.$$

3. The ratio of the probabilities is:

$$\frac{\frac{1}{2}}{\frac{1}{2 \times 5}} = 5.$$

[12] **point(s) — part c:** Consider the following extremely simple model: The model $f(\mathbf{x}; \boldsymbol{\theta})$ has only a single real-valued parameter $\boldsymbol{\theta} = b$ where $b \in \mathbb{R}$. Regardless of the input, the model always outputs b , i.e., $f(\mathbf{x}; \boldsymbol{\theta}) = b$ for all $\mathbf{x}$. Suppose that we have $N = 5$ samples. Moreover, the labels are

$$y^{(1)} = 3, y^{(2)} = 6, y^{(3)} = 8.5, y^{(4)} = 11 \text{ and } y^{(5)} = 16.$$

Compute 2 iterations of (full batch) gradient descent on the empirical risk/loss function

$$J(b) = \frac{1}{N} \sum_{i=1}^N |y^{(i)} - b|.$$

Use $b = 0$ as initialization and $\eta = 5$ as the step size. You can use the fact that the loss derivative is

$$\frac{\partial}{\partial b} |y - b| = \begin{cases} -1 & : \text{ if } b \leq y \\ 1 & : \text{ if } b > y \end{cases}.$$

Note: the step size $\eta = 5$ is rather large and is not typical in actual usage.

Iteration 1:

At $b = 0$, since $0 \leq y^{(i)}$ for all i , the derivative for each sample is -1 . Therefore, the average gradient is:

$$\frac{1}{N} \sum_{i=1}^N \frac{\partial}{\partial b} |y^{(i)} - b| = \frac{1}{5} (-1 - 1 - 1 - 1 - 1) = -1.$$

Update b :

$$b \leftarrow b - \eta \cdot (-1) = 0 - 5 \cdot (-1) = 5.$$

Iteration 2: At $b = 5$:

For $y^{(1)} = 3$, since $b > y^{(1)}$, the derivative is 1 . For $y^{(2)} = 6 \dots y^{(5)} = 16$, since $b \leq y^{(2)}, \dots, y^{(5)}$, the derivative is -1 .

Therefore, the average gradient is:

$$\frac{1}{N} \sum_{i=1}^N \frac{\partial}{\partial b} |y^{(i)} - b| = \frac{1}{5} (1 - 1 - 1 - 1 - 1) = \frac{-3}{5} = -0.6.$$

Update b :

$$b \leftarrow b - \eta \cdot (-0.6) = 5 - 5 \cdot (-0.6) = 5 + 3 = 8.$$

3 Relu network

[8] point(s) — **part a:** Let $g(u) = \max\{0, u\}$ be the relu activation. Plot over $x \in [-2, 2]$ the function

$$f(x; \theta) := g(x + b_1) - g(-(x + b_2)) + b_3 \quad \text{where} \quad \theta = [b_1, b_2, b_3] = [-1, 1, 1]$$

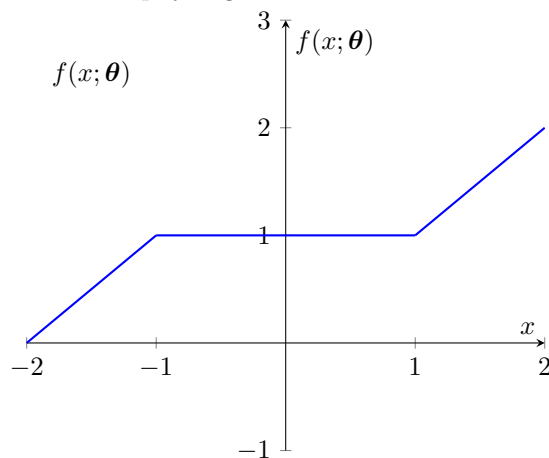
(Hint: It is piecewise linear function with 3 linear pieces.)

The first term $g(x + b_1) = g(x - 1)$ is the relu shifted right by 1 (so the elbow is at 1).

The second term $-g(-(x + b_2)) = -g(-x - 1)$ is the “180° rotated relu” shift left by 1 (so the elbow is at -1)

The third term is a constant at 1

Add them up, you get:



[12] **point(s) — part b:** Recall the i -th contribution to the empirical risk with the square error loss is defined as $J_i(\boldsymbol{\theta}) = (y^{(i)} - f(x^{(i)}; \boldsymbol{\theta}))^2$. Consider the following dataset with $N = 3$:

$$x^{(1)} = 2, \quad x^{(2)} = -2, \quad x^{(3)} = 0 \quad (2)$$

$$y^{(1)} = 5, \quad y^{(2)} = -1, \quad y^{(3)} = 2 \quad (3)$$

Compute the numerical values of $\frac{\partial J_i}{\partial \boldsymbol{\theta}}(\boldsymbol{\theta})$ for each $i = 1, 2, 3$, where $\boldsymbol{\theta}$ is as in part a. Show all work. Additional space on the next page available. (Hint: if your answer is correct, then you should have

$$\frac{\partial J}{\partial b_1}(\boldsymbol{\theta}) = -2, \quad \frac{\partial J}{\partial b_2}(\boldsymbol{\theta}) = \frac{2}{3}, \quad \frac{\partial J}{\partial b_3}(\boldsymbol{\theta}) = -2 \quad (4)$$

where $J(\boldsymbol{\theta}) = \frac{1}{3} \sum_{i=1}^n J_i(\boldsymbol{\theta})$.

Let $\hat{y}^{(i)} = f(x^{(i)}; \boldsymbol{\theta})$. From the graph, we see that

$$\hat{y}^{(1)} = 2, \quad \hat{y}^{(2)} = 0, \quad \hat{y}^{(3)} = 1.$$

Now,

$$\frac{\partial J_i}{\partial b_j} = -2(y^{(i)} - \hat{y}^{(i)}) \frac{\partial \hat{y}^{(i)}}{\partial b_j}.$$

Note that

$$-2(y^{(1)} - \hat{y}^{(1)}) = -2(5 - 2) = -6 \quad -2(y^{(2)} - \hat{y}^{(2)}) = -2(-1 - 0) = 2 \quad -2(y^{(3)} - \hat{y}^{(3)}) = -2(2 - 1) = -2$$

Case when $j = 3$, we have $\partial \hat{y}^{(i)} / \partial b_j = 1$ for every $i = 1, 2, 3$. Hence

$$\frac{\partial J_1}{\partial b_3} = -6 \quad \frac{\partial J_2}{\partial b_3} = 2 \quad \frac{\partial J_3}{\partial b_3} = -2$$

Case when $j = 2$, only the $-g(-x - b_2)$ term depends on b_2 .

$$\frac{\partial}{\partial b_2}(-g(-x - b_2)) = \begin{cases} 1 & : b_2 < -x \\ 0 & : b_2 > -x \end{cases}$$

Thus

$$\partial \hat{y}^{(i)} / \partial b_2 = 1$$

is nonzero only for $i = 2$. Hence

$$\frac{\partial J_1}{\partial b_2} = 0 \quad \frac{\partial J_2}{\partial b_2} = 2 \quad \frac{\partial J_3}{\partial b_2} = 0$$

Case when $j = 1$, only the $g(x + b_1)$ term depends on b_1 .

$$\frac{\partial}{\partial b_1}(g(x + b_1)) = \begin{cases} 1 & : b_1 > -x \\ 0 & : b_1 < -x \end{cases}$$

Thus

$$\partial \hat{y}^{(i)} / \partial b_1 = 1$$

is nonzero only for $i = 1$.

Hence

$$\frac{\partial J_1}{\partial b_1} = -6 \quad \frac{\partial J_2}{\partial b_1} = 0 \quad \frac{\partial J_3}{\partial b_1} = 0$$

Putting it all together

$$\frac{\partial J}{\partial b_1} = \frac{1}{3}(-6 + 0 + 0) = -2, \quad \frac{\partial J}{\partial b_2} = \frac{1}{3}(0 + 2 + 0) = \frac{2}{3}, \quad \frac{\partial J}{\partial b_3} = \frac{1}{3}(-6 + 2 - 2) = -2.$$

[8] **point(s)** — **part c:** Calculate, showing work, θ after 1 gradient descent update with step size $\eta = 3/4$. Plot over $x \in [-2, 2]$ the function $f(x; \theta)$ with the updated parameters. (Hint: you should get $[0.5, 0.5, 2.5]$)

$$b_1 \leftarrow b_1 - (3/4)(-2) = -1 + 3/2 = 1/2 = 0.5$$

$$b_2 \leftarrow b_2 - (3/4)(2/3) = 1 - 1/2 = 1/2 = 0.5$$

$$b_3 \leftarrow b_3 - (3/4)(-2) = 1 + 3/2 = 5/2 = 2.5$$

The first term $g(x + b_1) = g(x + 0.5)$ is the relu shifted right by 0.5 (so the elbow is at 0.5).

The second term $-g(-(x + b_2)) = -g(-(x + 0.5))$ is the “180° rotated relu” shift left by 0.5 (so the elbow is at 0.5).

Thus, the sum of the first and second term is

$$x + 0.5.$$

Conclusion: The third term is 2.5 so the overall function is

$$\boxed{x + 3}$$

from which you should be able to plot easily.

4 Tensor manipulation

[8] point(s) — **part a:** Write the tensor produced by the code block below:

```
import numpy as np
y = np.array([0,0,1,2,2,2])
(1.0*(y[:,np.newaxis] == np.arange(3))).T
```

(You can round your answer. You don't have to format the output like Python does. You don't need to show your work. However, if something is wrong, you need to show work to receive partial credit.)

```
[[1, 1, 0, 0, 0, 0],
 [0, 0, 1, 0, 0, 0],
 [0, 0, 0, 1, 1, 1]]
```

[8] point(s) — **part b:** Write the tensor produced by the code block below:

```
import numpy as np
n_samples, n_context = 3, 2
softmaxKq = np.array([[1,2],[3,4],[5,6]])
softmaxKq_diag = np.expand_dims(softmaxKq, axis=1) * np.eye(n_context)
softmaxKq_diag[1,:,:]
```

(You can round your answer. You don't have to format the output like Python does. You don't need to show your work. However, if something is wrong, you need to show work to receive partial credit.)

```
[[3, 0],
 [0, 4]]
```